Linq

Language-Integrated Query is .NET component that allows for standard unified query language.

Written in C# and has many providers.
- Linq to Objects : System.Linq - arrays, lists etc...
- Linq to SQL : System.Data.Linq - MSQL
- Linq to Entities : System.Data.Entity - EF
- Linq to XML : System.xml.Linq - XML

Benefits
- Intuitive syntax
- type safety
- Integrated in C# and .NET
- ORM integration (Entity Framework)

Linq Syntax

Linq queries can be written for class that implement the IEnumerable<T> and IQueryable<T> interfaces.

IEnumerable<T>: implemented by all built in collections

IQueryable<T>: implements extension methods

Both included in System.Linq

Query Operators

Projection : Select, SelectMany, Index
Filtering : Where, OfType, Index
Grouping : GroupBy
Sorting : OrderBy, OrderBy Descending
Aggregation : Aggregate, AggregateBy, Average, Count, CountBy
Concatenation : Concat
Qualifiers : All, Any, Contains
Join : Join, GroupJoin
Set : Distinct, Except, Union

Query Syntax

Similar to SQL : From object in source
where condition
select object

- Start with from in source
- then a filter, sort etc condition
- Lastly a projection select

```
from <range variable> in IEnumerable<T> or IQueryable<T> Collection
<Standard Query Operators> <expression>
<select or groupBy operator> <result>
```

Code

```
// In memory array object
int[] numArr = new[] { 1, 2, 3, 10, 15, 8 };
filterNums(numArr);
```

```
public static void filterNums(int[] arr)
{
    // Uses the IEnumerable interface, which is an array built-in
    // Here we are using Linq query syntax to filter numbers with a WHERE clause
    IEnumerable<int> result = from nums in arr
                              where nums < 10
                              select nums;

    StringBuilder sb = new();

    foreach (int n in result)
    {
        sb.AppendLine($"Number: {n}");
    }

    Console.WriteLine(sb.ToString());
}
```

```
$ dotnet run
Number: 1
Number: 2
Number: 3
Number: 8
```

Method Syntax

Allows us to use extension methods of IEnumerable<T> and IQueryable<T> implemented classes based on a Lambda expression.

Collection.<Extension Method>(Lambda expression)

```
// Result is implicitly inferred here to IEnumerable<int>
var result = arr.Where(num => num > 5).ToList();
```

```
var result = gamesArr.Where(game => game.Stock > 5).ToList<Game>();
```